«Национальный исследовательский университет
«Высшая школа экономики»»

Лицей

Индивидуальная выпускная работа

Сканер аллергенов «NutriScan»

Выполнил Сафронов Ярослав Михайлович

Москва 2026

1. Введение

NutriScan – это android-приложение, которое позволяет пользователям задать собственный список аллергенов и при добавлении продукта (через сканирование штрих-кода или поиск) сразу получать информацию о наличии или отсутствии опасных ингредиентов в его составе. Кроме того, данное приложение помогает контролировать состав пищи и придерживаться диеты благодаря встроенному подсчёту КБЖУ и возможности вести дневник съеденных продуктов. Я пришел к идее создания этого проекта, так как на личном опыте убедился, насколько трудно читать длинный состав продукта, часто написанный мелким шрифтом. По мере увеличения количества товаров требуется тратить всё больше времени на чтение калорийности и состава, и это очень изматывает.

2. Проблемное поле

Многие люди сталкиваются с пищевой аллергией или непереносимостью отдельных компонентов продуктов и нуждаются в возможности быстро определять, безопасен ли конкретный товар. На практике это сложно: состав часто напечатан мелким шрифтом, а ингредиенты могут указываться под синонимичными названиями, из-за чего пользователю трудно распознать наличие опасного вещества. В результате проверка состава занимает много времени и иногда может привести к ошибке.

На данный момент в российском сегменте практически отсутствуют удобные и доступные инструменты, которые помогали бы людям с аллергией быстро и надёжно проверять продукты в повседневной жизни. Существующие зарубежные приложения, как правило, не поддерживают российские базы данных и не распознают составы на русском языке, что делает их использование затруднительным.

На момент написания отчёта я сделал опрос в виде онлайн-формы для проведения количественного исследования. К сожалению, сейчас там ещё менее 10 респондентов, но уже можно сделать определённые выводы. Некоторые участники отметили, что они сталкивались с ситуациями, когда нежелательный компонент обнаруживался в продукте уже после его покупки. На данный момент пользователи вынуждены изучать состав, читая текст с упаковки, или отправляя товар искусственному интеллекту. Удобство этого процесса оценивается

ими на 3 балла из 5. Подобные результаты показывают, что разрабатываемый продукт будет востребован не только среди людей с аллергией, но и среди тех, кто просто избирателен в еде и хочет быстро, безошибочно проверять состав покупок. Также участники опроса отметили, что следят за калорийностью съеденных продуктов.

3. Целевая аудитория

Продукт может заинтересовать широкую аудиторию, поскольку рассчитан на пользователей, сталкивающихся с проблемой пищевой аллергии или просто желающих контролировать калорийность и содержание своего рациона. К выделяемым группам относятся:

- Подростки и молодые люди (14 – 25 лет), осознанно следящие за своим рационом или имеющие аллергические реакции на определённые продукты.
- Взрослые пользователи, страдающие пищевой аллергией, непереносимостью отдельных ингредиентов (например, глютена, орехов и т. п.) или желающие контролировать калорийность продуктов.
- Родители детей с аллергией, которым важно быстро проверять состав продуктов перед покупкой или употреблением.

Для всех указанных групп характерна потребность в простом и надёжном инструменте, который позволяет сократить время анализа состава продукта и избежать ошибок при чтении маркировки. Кроме того, приложение будет полезно людям, контролирующим калорийность питания или придерживающимся специальных диет – благодаря встроенному подсчёту энергетической ценности и возможности вести дневник безопасных продуктов.

4. Функциональные требования

В первую очередь приложение позволяет сканировать штрих-код через камеру устройства или выполнять ручной поиск по нему, после чего сервер будет сравнивать состав продукта со списком аллергенов пользователя и отправлять

информацию о статусе безопасности для конкретного человека. Также поддерживается самостоятельное добавление товара, если окажется, что его нет в базе (необходимо указать название, бренд, состав, КБЖУ). У пользователя есть возможность менять настройки и параметры: системное имя, пароль, список аллергенов, ежедневный лимит калорий. В случае, если продукт не имеет штрихкода, пользователь сможет добавить его вручную, указав все параметры. При этом товар сохранится с внутренним идентификатором. Также пользователь может вести дневник питания (доступны 4 категории приёма пищи), есть возможность просматривать статистику КБЖУ за каждый день, смотреть прогресс калорий относительно установленного дневного лимита потребления. Есть доступ к последним отсканированным или найденным через поиск товарам во вкладке "История".

5. Стек технологий

Мобильное приложение разрабатывается на Kotlin с архитектурой Model-View-ViewModel и использованием фреймворка Jetpack Compose для создания интерфейса. Для сетевого взаимодействия используется Retrofit, а для внедрения зависимостей - Koin. За работу с камерой отвечает CameraX, анализ изображений выполняется через Google ML Kit.

Серверная часть разрабатывается на Python с использованием FastAPI. Для валидации входящих данных и сериализации ответов сервера использован Pydantic. В качестве ORM для работы с базой данных (модели, запросы, связь с БД) используется SQLAlchemy. Запуск сервера и прослушивание сетевых запросов происходит при помощи uvicorn.

В качестве базы данных используется PostgreSQL, а для выдачи TLS-сертификатов и шифрования - Caddy. Серверная часть запускается при помощи Docker Compose.

6. Этапы работы над проектом

1) Декабрь - Создание основы мобильного приложения. Настройка проекта в Android Studio, создание главного экрана и базовой навигации между экранами. Формирование первоначальной структуры приложения.

2) Январь - реализация сканирования и серверной части. Разработка основного функционала сканирования штрих-кодов с использованием камеры теле-

фона. Создание backend-части проекта, тестовой базы данных, базового API и интеграции с PostgreSQL.

3) Февраль - интеграция мобильного приложения с API. Подготовка приложения к работе с сервером и реализация сетевого взаимодействия через Retrofit. Добавление запроса для получения информации о товаре по штрих-коду. Доработка структуры данных товаров и подготовка API для дальнейшего расширения функциональности.

4) Март - разработка авторизации, профиля и системы аллергенов. Реализация экранов регистрации и авторизации, валидации полей и сохранения токена пользователя. Создание профиля пользователя и соответствующих серверных методов.

5) Апрель - структурирование и подготовка бэкенда. Переработка структуры проекта, настройка docker compose. Подготовка серверной части к дальнейшему развёртыванию.

6) Июль и август - развитие функциональности приложения. Доработка экрана выбора аллергенов. Был реализован функционал контроля калорийности: создан отдельный экран, сделано получение и изменение дневного лимита калорий, исправлен ввод данных. Разработка истории сканирований на стороне сервера и подключение её к приложению. Добавление возможности обработки ненайденных товаров и загрузки истории с сервера. Также реализован экран добавления новых товаров и соответствующий метод на сервере.

7) Сентябрь - расширение функционала аллергенов и дневника питания. Внедрение словаря аллергенов и алгоритма поиска по ингредиентам и словам-триггерам. Глобальная переработка экрана "Дневник питания" под работу с сервером (добавление диаграммы БЖУ, статистики). Исправление логики работы сканера (циклическая навигация, задержки). Реализация изменения имени пользователя и пароля, добавление диалоговых окон. Подготовка итоговых пользовательских сценариев и проектной заявки для защиты.

8) Октябрь - финальная доработка, настройка CI и покрытие тестами. Добавление инструмента миграций (Alembic), упрощение структуры БД, разделение истории и продуктов на бэкенде. Внедрение refresh-токенов для поддержания сессии. Добавление валидации, ручного ввода штрих-кодов, расчета контрольных сумм и поддержки товаров без штрих-кода.

9) Ноябрь (начало): Покрытие проекта тестами: настройка JUnit, тестирова-

ние auth, products, diary, history на бэкенде. Финальные исправления багов (кликабельность карточек в истории и дневнике, игнорирование сохранения скана) и добавление моковых данных.

7. Краткий анализ аналогичных продуктов

Анализ существующих решений показал, что зарубежные приложения для сканирования состава в большинстве случаев не поддерживают российские базы штрих-кодов и не работают с русскоязычными описаниями ингредиентов. Отдельные "трекеры здоровья" фокусируются на КБЖУ и активности, но не предоставляют персональной проверки аллергенов.

Похожие продукты:

1) OpenFoodFacts: имеет похожую с моим проектом цель, но он не имеет большинства российских продуктов в своей базе и не позволяет вести дневник питания и учитывать калорийность. У него достаточно перегруженный интерфейс, который лишь частично переведён на русский язык.

2) My Food Allergies Scanner – это приложение работает полностью на английском языке, имеет встроенную рекламу и ограниченный функционал в бесплатной версии. Тут вообще нет функционала ведения статистики или истории питания.

3) Mira – приложение, которое позиционирует себя как независимый гид на базе ИИ и сканер еды и косметики. Однако пользователи в отзывах отмечали субъективность 100-балльной оценки продуктов по формуле, в которой качественный продукт может получить низкий балл из-за алгоритмов расчёта. Также это приложение предлагает "более чистые и безопасные" альтернативы, и существует риск, что такое продвижение не гарантирует честное сравнение и может продвигать продукты спонсоров. Также без подписки ограничена история и поиск.

4) FatSecret – это приложение позволяет вести дневник питания и рассчитывать КБЖУ, имеет в базе часть российских продуктов. Но при этом оно не имеет базы аллергенов и составов, что усложняет контроль питания. Также отсутствует история сканирований.

8. Рефлексия

8.1. Проблемы, которые возникли в процессе работы над проектами и способы их решения

Проблемы с отступами и навигацией в приложении. Из-за постепенной работы над разными частями приложения возникли проблемы со вложенными Scaffold, которые давали двойные отступы сверху и снизу. Было решено задать отступ централизованно на уровне NavHost. Также двойные нажатия на кнопки открывали несколько экранов или приводили к излишнему сбросу стека, пришлось реализовывать защиту от повторного перехода.

Работа с камерой на старых Android. Также я тестировал приложение на телефоне с Android 8 и на нём не удавалось запустить анализ изображений (по всей видимости, из-за слишком слабого процессора или нехватки оперативной памяти). Для подобных ситуаций я добавил ручной ввод штрих-кода, чтобы не лишать пользователей возможности найти товар на слабых устройствах.

Ручная передача зависимостей во ViewModel привела к увеличению объёма кода и некоторым ошибкам, так что было принято решение внедрить Koin, чтобы централизованно управлять созданием репозиториев, автоматически разрешать зависимости.

Проблемы при тестировании бэкенда. У меня долго не получалось настроить тестовую БД, так как сервер при запуске тестов пытался сразу подключиться к PostgreSQL и выдавал ошибку исполнения. Удалось решить проблему переноса кода в lifespan, который игнорируется тестами.

8.2. Умения и навыки, приобретённые в процессе работы над проектом

Я улучшил свои знания в области разработки интерфейса на Jetpack Compose, работал с архитектурой ViewModel + Repository и изучил концепцию DI.

В области бэкенда первый раз пробовал фреймворк FastAPI и в целом работу с базой данных postgres, в которой больше одной таблицы. Разобрался подробнее в работе jwt и инструментов миграций.

Также немалый вклад внесла работа с системой контроля версий, Docker, GitHub Actions (с ними я вообще не работал до этого проекта, только знал, что они существуют). В будущем навыки настройки деплоя и тестов помогут более

качественно настроить следующие проекты и быстрее найти возможные ошибки.

По мере продвижения работы требовались базовые навыки дизайна, так что я освоил векторную графику в Figma (иконки приложения и аллергенов).

8.3. Что можно было бы сделать по-другому

Оглядываясь назад, я бы однозначно изменил некоторые моменты разработки. Во время реализации приложения я не проектировал структуру базы данных и по мере написания кода приходилось часто менять схемы, что приводило к путанице и расхождениям в требованиях к приложению и бэкенду. Сейчас я понимаю, что стоило с самого начала подключить Alembic для описания миграций, так как это бы убрало проблемы со сложным откатом изменений и падением сервера.

А вот в приложении у меня была противоположная ситуация. На начальных этапах я не использовал сервер, а писал экраны для работы с временными репозиториями. На разработку ушло немалое количество времени, а в итоге всё равно пришлось менять структуру классов и переиспользуемых компонентов. Если бы писал сейчас, то просто не начинал бы писать экран, пока не сделано хотя бы минимальное сетевое взаимодействие.

Также я считаю, что стоило сразу начать писать тесты параллельно с разработкой, так как они бы смогли найти баги в логике без трудного выискивания ошибок глазами.

8.4. Идеи для дальнейшего развития проекта

Мне кажется, что было бы неплохо сделать поиск продуктов по названию, а не только по штрих-коду. Это смогло бы решить проблему товаров, у которых нет кода на упаковке (сейчас их можно делать, но другие пользователи не смогут найти их, так как не знают UUID). Также это даст возможность находить товары других пользователей. Можно добавить редактирование и удаление записей дневника, статистику за неделю и месяц, возможность удалять записи из истории. Также будет хорошей практикой реализация сохранения данных на телефоне для работы без интернета. Ещё можно сделать сброс пароля по почте, уведомления. И в качестве финальной доработки я бы улучшил стратегию

поиска аллергенов в составе продукта. Скорее всего, лучшим вариантом будет размещение локальной ИИ-модели на сервере для обработки текста, так как встречаются достаточно запутанные формулировки, которые трудно распознавать обычным поиском подстроки в составе.